# README Sprint S06 · Datos listos para modelar

**Pregunta guía:** ¿están los datos en condiciones de entrenar algo fiable?
**Fechas:** 6 y 9 de noviembre · **Horas:** 12 h (3 h teoría · 5 h práctica · 4 h PBL)

📄 **Para imprimir:** todos los documentos del sprint en PDF, a color y ecofriendly, están juntos en la carpeta `pdf/` (descárgalos para que funcionen los enlaces).

> **Sigue el orden de arriba abajo y abre solo lo que pone en cada fila.** Los demás documentos del sprint son para consultar si te atascas (apartado «Si te atascas»). Si el tiempo aprieta, haz solo los ejercicios marcados «en clase»: el resto es para casa.

## Qué aprenderás

1. Explorar una tabla con pandas y elegir el gráfico adecuado para cada tipo de dato.
2. Encontrar defectos en un dataset (nulos, duplicados, *outliers*, tipos y fechas mal formados, categorías inconsistentes) y **razonar de dónde vienen**.
3. Construir un **pipeline** reproducible (`ColumnTransformer` + `Pipeline`) que impute, escale y codifique sin fuga de datos.
4. Medir un modelo con honestidad: *baseline*, métricas, validación cruzada, partición temporal.
5. Reconocer y evitar la **fuga de datos** (*data leakage*).

## Día 1 · Viernes 6/11

| ☐ | Qué haces | Abre | Min |
|---|---|---|---|
| ☐ | 🔔 Ceremonia · Planning del reto «Datos sucios, cliente impaciente» y reparto de roles | — | 10 |
| ☐ | 📖 Teoría · pandas y EDA: mirar los datos antes de tocarlos | Presentación S06_01 | 20 |
| ☐ | 💻 Práctica · pandas, EDA y primer chequeo de calidad (los ejercicios marcados «en clase») | Cuaderno S06_01 | 55 |
| ☐ | 🎯 Reto · Arrancar el inventario de defectos del fichero sucio (pasos 1–2) | Reto en una página | 30 |
| ☐ | 📖 Teoría · Limpieza de datos: duplicados, tipos, fechas, categorías y nulos | Presentación S06_02 | 30 |
| ☐ | 💻 Práctica · Limpieza: duplicados, tipos, fechas, categorías, imposibles y nulos (los ejercicios marcados «en clase») | Cuaderno S06_02 | 55 |

**En clase: 3 h 20 min.**

**Al acabar el día, deberías poder decir:**

☐ Mi equipo tiene el inventario de defectos del fichero sucio.
☐ Sé limpiar duplicados, tipos y fechas con pandas.

## Día 2 · Lunes 9/11

| ☐ | Qué haces | Abre | Min |
|---|---|---|---|
| ☐ | 💻 Práctica · Escalado, codificación, `ColumnTransformer` y `Pipeline` (los ejercicios marcados «en clase») | Cuaderno S06_03 · Presentación S06_03 | 40 |
| ☐ | 🎯 Reto · Función de limpieza, partición train/test, *pipeline* y *baseline* (pasos 3–4) | Reto en una página | 30 |
| ☐ | 📖 Teoría · Métricas y validación: *baseline*, validación cruzada y fuga de datos | Presentación S06_04 | 30 |
| ☐ | 💻 Práctica · *Baseline*, métricas, validación cruzada y fuga de datos (los ejercicios marcados «en clase») | Cuaderno S06_04 | 45 |
| ☐ | 🎯 Reto · Informe, README y ensayo (paso 5) | Reto en una página | 20 |
| ☐ | 🔔 Ceremonia · Entrega en `sprint-06/`, autoevaluación y coevaluación, review y retrospectiva | — | 35 |

**En clase: 3 h 20 min.**

**Al acabar el día, deberías poder decir:**

☐ Sé encontrar los defectos de un fichero sucio y razonar de dónde vienen.
☐ Sé montar un *pipeline* que imputa, escala y codifica sin fuga de datos.
☐ Sé comprobar con un *baseline* y validación cruzada si un modelo mejora de verdad.

## El reto en pocas palabras

**Qué os pide el cliente.** TurisData os envía por correo un fichero de reservas exportado **a mano desde varios sistemas** (`reservas_sucio.csv`). El gerente quiere un primer modelo de cancelaciones «para ayer», pero **no se fía de los datos** (con razón). Os pide dos cosas:

1. Un **informe corto** que diga **qué está mal en los datos y por qué**.
2. Un **proceso automático** que deje los datos listos cada vez que le llegue un fichero así, con un primer modelo sencillo de referencia (un *baseline*).

**Qué vais a entregar:** un repositorio o carpeta con `informe_calidad.md`, `reto_S06.ipynb` y `README.md`.

**Núcleo y extra.** Núcleo: inventario de defectos, función de limpieza y *baseline* con su *pipeline*. Extra: informe más detallado y comparativa de variantes de limpieza.

Antes de empezar, probad la demo interactiva (muestra lo que tendréis al final). Para trabajar: reto en una página. Para entregar: qué se sube y rúbrica. Para consultar: ficha del reto (qué pide el cliente, ayudas y errores frecuentes) y cada paso explicado.

## ✅ Antes de cerrar el sprint (Definition of Done)

**Archivos subidos en el repositorio del equipo, carpeta `sprint-06/`** (marca cada uno cuando lo veas en GitHub): ☐ `informe_calidad.md` · lo creáis a partir de una plantilla
☐ `reto_S06.ipynb` · lo creáis
☐ `README.md` · lo modificáis
☐ `autoevaluacion_nombre.md` · **uno por persona** (la coevaluación va dentro del mismo fichero): plantilla

**Total en `sprint-06/`: 3 ficheros del equipo + 1 hoja por persona** (un equipo de 3 sube 6). Aparte: reservas_sucio.csv en `data/` si aún no está; los extras solo si los hacéis. La *review* y la *retro* no se suben.

**Comprobaciones antes de dar el sprint por cerrado:** ☐ Notebook ejecutable de principio a fin (`Restart & Run all`)
☐ El informe recoge cada defecto con causa probable y tratamiento
☐ Hay *baseline* (`DummyClassifier`) y un modelo, comparados con validación cruzada
☐ Alguien que **no escribió esa parte** ha revisado el entregable con la rúbrica y lo ha anotado en el propio entregable

## En casa (cuando puedas)

- Los ejercicios de cada cuaderno que **no** se hacen en clase. Se autocorrigen, y debajo llevan una «Solución de referencia» plegada por si te quedas atascado.
- Si quieres repasar la teoría con calma: apuntes del sprint.

## Si te atascas

1. Lee primero la parte «Si os atascáis» de la ficha completa del reto.
2. Mira el código explicado del cuaderno en el que estás.
3. Pregunta a tu equipo y, después, al docente. Si usas IA, cuenta en el README qué te dijo y qué comprobaste.

## Todo el material, por carpetas

Las carpetas están **numeradas en el orden en que las usas**. Cada documento está en `.md`.

| Carpeta | Qué hay |
|---|---|
| `1_Teoria/` | Las presentaciones del sprint y los apuntes para repasar |
| `2_Practica/` | Los cuadernos (Colab) y su explicación línea a línea en `codigo_explicado/` |
| `3_Reto/` | `1_Trabajar` (reto en una página, plantillas y datos) · `2_Entregar` (qué se sube, rúbrica y autoevaluación) · `3_Consultar` (ficha del reto y cada paso explicado) |
| `4_Referencia/` | Infografías del sprint |

Cómo se trabaja un reto, en todos los sprints: método de trabajo.

## Temario oficial de este sprint

| Unidad | Conceptos que entran |
|---|---|
| M2 · Exploración del conjunto de datos (UD 2.1 a 2.5) | Funciones del Data Scientist; datos estructurados, no estructurados y series temporales · read_csv, info, describe, filtrado, agrupación, uniones · matplotlib, seaborn, plotly; elección del gráfico según el tipo de dato · Nulos, duplicados, outliers, tipos incorrectos; detección de defectos y de sus causas · Escalado, codificación de categóricas, reducción de dimensionalidad, separación train/test |
| UD 3.1 | Flujo de trabajo con scikit-learn: estimadores, pipelines, validación cruzada |
| UD 3.2 | Métricas y evaluación: MAE/RMSE/$R^2$, precisión/recall/F1, ROC-AUC, sesgo-varianza, GridSearchCV |

**Y ahora:** Sprint 7 · ML supervisado: regresión y clasificación (martes 10/11).

*12 h (3 h teoría · 5 h práctica · 4 h PBL).*